

DEPARTAMENTO:	Ciencias de la Computación	CARRERA:	ITIN
ASIGNATURA:	Fundamentos de Sistemas Web	NIVEL:	4to
FECHA:	19/07/2026	DOCENTE:	Ing. Geovanny Brito
PRÁCTICA N°:	5	CALIFICACIÓN:	

LABORATORIO DE MANIPULACIÓN DE ELEMENTOS WEB

Diego Montesdeoca

El laboratorio es un sistema de registro de estudiantes construido como práctica de laboratorio de manipulación del DOM con JavaScript vanilla. Permite ingresar nombre, carrera y semestre de estudiantes a través de un formulario, almacenándolos en un arreglo global de objetos. Los datos se renderizan dinámicamente en una tabla HTML que incluye columna de acciones (ver JSON, editar, eliminar). Se implementaron funcionalidades de búsqueda por texto, filtro por carrera, modo edición con cambio de estado del botón principal, eliminación con confirmación visual, visualización de datos en formato JSON individual y general mediante un modal Bootstrap, y un contador animado. El proyecto integra tres librerías externas: SweetAlert2 para diálogos de confirmación, Toastify JS para notificaciones emergentes, y Anime.js v4 para animaciones de entrada/salida de elementos, transiciones del contador y efectos visuales en la interfaz. El diseño utiliza Bootstrap 5 con tema oscuro personalizado, iconos Font Awesome, y una paleta de colores con tono masculino y moderno.

Palabras clave: DOM, JavaScript, Toastify JS

Link de Github: https://github.com/damontesdeoca2-cpu/LabU3_MontesdeocaDiego.git

Tabla de Contenido

1. Descripción corta de la actividad	4
2. Objetivo general.....	4
3. Objetivos específicos	4
4. Marco conceptual.....	4
5. Desarrollo de las funcionalidades implementadas	6
5.1 Arreglo global y estructura del formulario.....	6
5.2 Registro de un estudiante en el arreglo	7
5.3 Actualización dinámica de la tabla	8
5.4 Columna de acciones: JSON, editar y eliminar	9
5.5 Modal de Bootstrap para visualización de JSON	10
5.6 Visualización de JSON individual y JSON general	11
5.7 Modo de edición de estudiantes	12
5.8 Eliminación de estudiantes con confirmación	13
5.9 Actualización del contador de estudiantes	14
5.10 Limpieza de tabla, arreglo y formulario	14
5.11 Mensajes visuales con Toastify.....	16
5.12 Búsqueda de estudiantes por texto.....	17
5.13 Filtro por carrera.....	18
5.14 Pruebas del sistema con estudiantes registrados	19
6. Librería externa: SweetAlert2 y Toastify.....	20
7. Criterio personal	20
8. Conclusiones	21
9. Recomendaciones	21
10. Bibliografía	21

Índice de figuras

Figura 1. Estructura de archivos del proyecto en el editor de código.

Figura 2. Interfaz inicial del sistema de registro de estudiantes, sin datos cargados.

Figura 3. Registro de un estudiante y notificación de confirmación mediante Toastify.

Figura 4. Modal de Bootstrap mostrando el JSON de un estudiante individual.

Figura 5. Modal de Bootstrap mostrando el JSON general de los estudiantes registrados.

Figura 6. Formulario cargado en modo edición, con el botón principal indicando "Guardar cambios".

Figura 7. Tabla actualizada tras confirmar la edición y notificación de éxito.

Figura 8. Cuadro de confirmación de SweetAlert2 antes de limpiar la tabla.

Figura 9. Tabla reconstruida con los estudiantes de prueba tras confirmar la operación.

Figura 10. Resultado de la búsqueda por texto sobre el arreglo de estudiantes.

Figura 11. Restablecimiento de la tabla completa tras limpiar el campo de búsqueda.

Figura 12. Filtro de estudiantes aplicado sobre la carrera Tecnologías de la Información.

Figura 13. Vista responsiva del formulario de registro en un ancho de pantalla reducido.

1. Descripción corta de la actividad

Repositorio GitHub: https://github.com/damontesdeoca2-cpu/LabU3_MontesdeocaDiego.git

Este laboratorio amplía el registro de estudiantes trabajado previamente en clase mediante manipulación del DOM. A partir de los archivos base index.html y app.js, se incorporaron nuevas funcionalidades para registrar estudiantes en un arreglo de objetos, mostrarlos dinámicamente en una tabla, visualizar la información en formato JSON a través de un modal de Bootstrap, editar y eliminar registros, filtrar y buscar estudiantes, y aplicar la librería externa Toastify para mejorar los mensajes visuales del sistema, junto con SweetAlert2 para las confirmaciones de acciones críticas.

La práctica refuerza el acceso y la modificación de elementos HTML, el manejo de eventos del DOM, la creación dinámica de nodos, la actualización de tablas en tiempo real, el uso de arreglos de objetos como fuente de datos y la presentación estructurada de información en formato JSON.

2. Objetivo general

Aplicar técnicas de manipulación del DOM mediante JavaScript para ampliar un sistema de registro de estudiantes, incorporando arreglos de objetos, modales de Bootstrap, edición y eliminación de registros, filtros de búsqueda y librerías externas de interacción visual.

3. Objetivos específicos

- Almacenar y gestionar los estudiantes registrados mediante un arreglo global de objetos.
- Actualizar dinámicamente la tabla HTML a partir del contenido del arreglo de estudiantes.
- Implementar un modal de Bootstrap para visualizar la información de los estudiantes en formato JSON.
- Incorporar funcionalidades de edición y eliminación de registros con confirmación visual.
- Desarrollar filtros de búsqueda por texto y por carrera sobre el arreglo de datos.
- Integrar las librerías externas SweetAlert2 y Toastify para mejorar la experiencia de usuario.
- Documentar el desarrollo con capturas, código y análisis de cada funcionalidad implementada.

4. Marco conceptual

DOM (Document Object Model): es una representación estructurada del documento HTML que el navegador construye al cargar una página. Cada etiqueta se convierte en un objeto que JavaScript puede leer, modificar, crear o eliminar. Gracias al DOM es posible cambiar textos, agregar filas a una tabla, mostrar mensajes, abrir componentes, modificar clases de Bootstrap y actualizar información sin recargar la página (MDN Web Docs, 2026).

Eventos y el evento click: un evento es una señal que el navegador emite cuando ocurre una interacción del usuario, como una pulsación de teclado o un clic del mouse. El evento click es el más utilizado en interfaces con botones, ya que permite ejecutar una función específica cada vez que el usuario presiona un elemento, mediante `addEventListener` (MDN Web Docs, 2026a).

getElementById: método del objeto document que localiza un elemento del DOM a partir de su atributo id, devolviendo una referencia directa que puede leerse o modificarse (MDN Web Docs, 2026c).

createElement y appendChild: createElement genera un nuevo nodo HTML en memoria (por ejemplo, una fila <tr> o una celda <td>), mientras que appendChild inserta ese nodo dentro de otro elemento existente, permitiendo construir la tabla de forma dinámica (MDN Web Docs, 2026d).

textContent e innerHTML: textContent asigna o lee texto plano dentro de un elemento, mientras que innerHTML permite insertar marcado HTML completo, útil para agregar íconos o etiquetas con formato dentro de una celda (MDN Web Docs, 2026e).

className y classList: className permite asignar directamente el valor completo de las clases CSS de un elemento, mientras que classList ofrece métodos como add, remove y toggle para modificar clases individuales sin afectar las demás, lo cual resulta útil al alternar el estado visual del botón principal entre modo agregar y modo edición (MDN Web Docs, 2026f).

JSON.stringify: convierte un objeto o arreglo de JavaScript en una cadena de texto con formato JSON. En este laboratorio se utiliza para mostrar, dentro del modal de Bootstrap, la información de un estudiante individual o del arreglo completo de estudiantes registrados, aplicando una indentación de dos espacios para mejorar la legibilidad (MDN Web Docs, 2025).

Modal de Bootstrap: componente interactivo que se superpone al contenido principal de la página para mostrar información o solicitar una acción al usuario. Bootstrap 5 lo implementa mediante la clase modal y se controla desde JavaScript con new bootstrap.Modal() y el método .show() (MDN Web Docs, 2022).

SweetAlert2: librería JavaScript que reemplaza las ventanas de confirmación nativas del navegador (confirm, alert) con diálogos personalizables, responsivos y accesibles (WAI-ARIA). Se utiliza en este proyecto para confirmar la eliminación de estudiantes y la limpieza de la tabla (SweetAlert2, 2021).

Toastify JS: librería ligera de JavaScript puro que genera notificaciones emergentes (toasts) con diseño personalizable, posición configurable y duración ajustable. Se aplica en este proyecto para informar al usuario sobre acciones como agregar, editar, eliminar o buscar registros (Apvarun, 2020).

Anime.js: motor de animaciones JavaScript que permite transiciones fluidas en elementos del DOM mediante una API declarativa. En su versión 4 ofrece funciones como animate(), stagger() y createTimeline(). Se integra en este proyecto para animar la entrada de la tarjeta principal, las filas de la tabla, el contador numérico, efectos de sacudida en validaciones y la apertura del modal de JSON (Anime.js, 2026).

5. Desarrollo de las funcionalidades implementadas

A continuación, se documenta cada una de las funcionalidades desarrolladas sobre la base del registro de estudiantes construido en clase, incluyendo el código relevante, las capturas del resultado obtenido y el análisis correspondiente.

5.1 Arreglo global y estructura del formulario

Se definió un arreglo global llamado `estudiantesRegistrados` que funciona como fuente principal de datos del sistema. Junto a este se declararon variables de control para el contador de estudiantes y el índice del registro en edición. El formulario de entrada captura nombre, carrera y semestre mediante los elementos con id nombre, carrera y semestre.

```
const estudiantesRegistrados = [];  
let contadorEstudiantes = 0;  
let editandoIndex = null;  
  
const btnAgregar = document.getElementById("btnAgregar");  
const btnLimpiar = document.getElementById("btnLimpiar");  
const btnBuscar = document.getElementById("btnBuscar");  
const btnLimpiarBusqueda = document.getElementById("btnLimpiarBusqueda");  
const btnFiltrar = document.getElementById("btnFiltrar");  
const btnVerJSON = document.getElementById("btnVerJSON");  
  
btnAgregar.addEventListener("click", agregarEstudiante);  
btnLimpiar.addEventListener("click", limpiarTabla);  
btnBuscar.addEventListener("click", buscarEstudiantes);  
btnLimpiarBusqueda.addEventListener("click", limpiarBusqueda);  
btnFiltrar.addEventListener("click", filtrarPorCarrera);  
btnVerJSON.addEventListener("click", mostrarModalJSONGeneral);
```

Análisis: cada botón de la interfaz se enlaza a su función correspondiente mediante `addEventListener`, de modo que el evento click dispara la lógica de negocio sin necesidad de atributos `onclick` en el HTML, siguiendo una separación clara entre estructura y comportamiento.

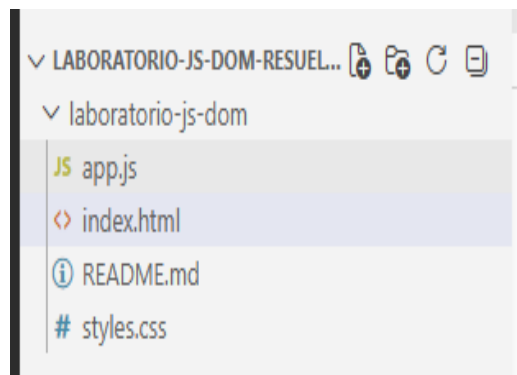


Figura 1. Estructura de archivos del proyecto en el editor de código.

5.2 Registro de un estudiante en el arreglo

La función `agregarEstudiante` obtiene los datos del formulario, valida que estén completos y, según el estado de `editandoIndex`, decide si el registro se agrega como nuevo elemento del arreglo o si reemplaza a un estudiante existente. Cuando la validación falla, se muestra una notificación de advertencia y se ejecuta una animación de sacudida sobre el botón principal.

```
function agregarEstudiante() {
  const estudiante = obtenerDatosFormulario();

  if (!validarEstudiante(estudiante)) {
    mostrarToast("Debe completar todos los campos.", "warning");
    return;
  }

  if (editandoIndex !== null) {
    estudiantesRegistrados[editandoIndex] = estudiante;
    editandoIndex = null;
    btnAgregar.innerHTML = '<i class="fas fa-plus me-1"></i>Agregar estudiante';
    btnAgregar.classList.remove("btn-warning-custom");
    btnAgregar.classList.add("btn-accent");
    mostrarToast("Estudiante editado correctamente.", "info");
  } else {
    contadorEstudiantes++;
    estudiante.id = contadorEstudiantes;
    estudiantesRegistrados.push(estudiante);
    mostrarToast("Estudiante agregado correctamente.", "success");
  }

  renderizarTabla(estudiantesRegistrados);
  actualizarTotal();
  limpiarFormulario();
}
```

Análisis: la variable `editandoIndex` actúa como bandera de estado. Cuando es `null`, el sistema interpreta que se trata de un registro nuevo; cuando contiene un número, la función reemplaza el objeto correspondiente en esa posición del arreglo, evitando duplicar registros durante la edición.

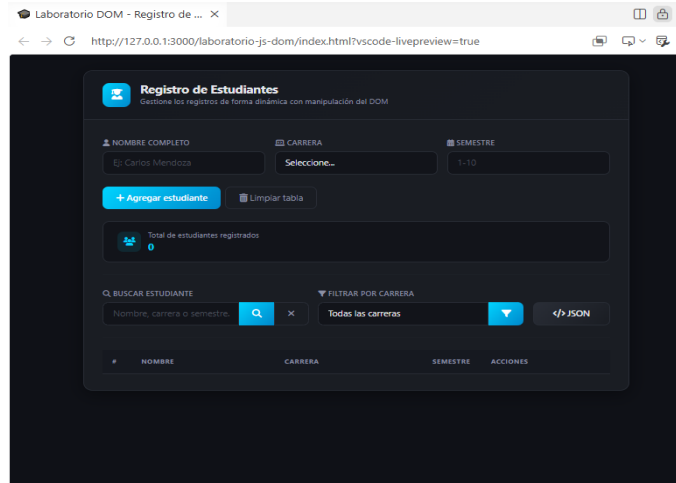


Figura 2. Interfaz inicial del sistema de registro de estudiantes, sin datos cargados.

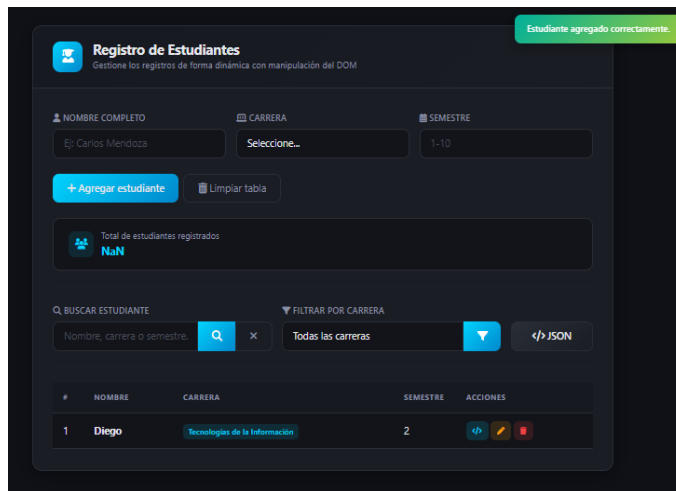


Figura 3. Registro de un estudiante y notificación de confirmación mediante Toastify.

5.3 Actualización dinámica de la tabla

La función `renderizarTabla` recibe una lista de estudiantes y reconstruye el cuerpo de la tabla utilizando `createElement` y `appendChild`. Por cada estudiante se generan las celdas de número, nombre, carrera, semestre y una columna de acciones con tres botones: ver JSON, editar y eliminar.

```
function renderizarTabla(lista) {
  const cuerpoTabla = document.getElementById("cuerpoTabla");
  cuerpoTabla.innerHTML = "";

  if (lista.length === 0) {
    const emptyRow = document.createElement("tr");
    const emptyCell = document.createElement("td");
    emptyCell.colSpan = 5;
    emptyCell.innerHTML = '<div class="empty-state">No hay estudiantes registrados</div>';
    emptyRow.appendChild(emptyCell);
  }
}
```



```

    cuerpoTabla.appendChild(emptyRow);
    return;
}

lista.forEach((estudiante, index) => {
    const fila = document.createElement("tr");

    const columnaNumero = document.createElement("td");
    columnaNumero.textContent = index + 1;

    const columnaNombre = document.createElement("td");
    columnaNombre.innerHTML = '<strong>' + estudiante.nombre + '</strong>';

    const columnaCarrera = document.createElement("td");
    columnaCarrera.innerHTML = '<span class="badge-campus">' + estudiante.carrera +
'</span>';

    const columnaSemestre = document.createElement("td");
    columnaSemestre.textContent = estudiante.semestre;

    fila.appendChild(columnaNumero);
    fila.appendChild(columnaNombre);
    fila.appendChild(columnaCarrera);
    fila.appendChild(columnaSemestre);
    cuerpoTabla.appendChild(fila);
});
}

```

Análisis: la tabla nunca se edita directamente; en cada llamada se limpia el contenido con `innerHTML = ""` y se reconstruye a partir del arreglo, lo que garantiza que la vista siempre refleje el estado real de los datos. Cuando la lista está vacía, se muestra una fila con un mensaje informativo.

5.4 Columna de acciones: JSON, editar y eliminar

Dentro del ciclo de renderizado se crean tres botones por cada fila, cada uno con su propio listener de clic que recibe el índice o el objeto del estudiante correspondiente.

```

const columnaAcciones = document.createElement("td");

const btnVerJSON = document.createElement("button");
btnVerJSON.innerHTML = '<i class="fas fa-code"></i>';
btnVerJSON.className = "action-btn action-btn-json me-1";
btnVerJSON.addEventListener("click", function () {
    mostrarModalJSONIndividual(estudiante);
});

const btnEditar = document.createElement("button");
btnEditar.innerHTML = '<i class="fas fa-pen"></i>';
btnEditar.className = "action-btn action-btn-edit me-1";

```

```

btnEditar.addEventListener("click", function () {
    editarEstudiante(index);
});

const btnEliminar = document.createElement("button");
btnEliminar.innerHTML = '<i class="fas fa-trash"></i>';
btnEliminar.className = "action-btn action-btn-delete";
btnEliminar.addEventListener("click", function () {
    eliminarEstudiante(index);
});

columnaAcciones.appendChild(btnVerJSON);
columnaAcciones.appendChild(btnEditar);
columnaAcciones.appendChild(btnEliminar);
fila.appendChild(columnaAcciones);

```

Análisis: cada botón se construye por completo con JavaScript, incluyendo su clase CSS y su ícono de Font Awesome mediante innerHTML, y se le asigna un listener independiente que captura el índice de la fila en el momento de su creación, evitando así que las tres acciones se confundan entre sí.

5.5 Modal de Bootstrap para visualización de JSON

Se construyó un modal de Bootstrap identificado como modalJSON, que contiene un bloque <pre> con id contenidoJSON donde se inserta el texto generado por JSON.stringify. El modal se abre desde JavaScript mediante la clase bootstrap.Modal.

```

<div class="modal fade" id="modalJSON" tabindex="-1">
  <div class="modal-dialog modal-lg modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="modalJSONLabel">JSON de estudiantes</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <pre id="contenidoJSON" class="json-display"></pre>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-outline-custom" data-bs-
dismiss="modal">Cerrar</button>
      </div>
    </div>
  </div>
</div>

```

Análisis: el modal es un único componente reutilizable para dos escenarios distintos (JSON individual y JSON general); su título y contenido se actualizan dinámicamente antes de invocar el método `show()`, en lugar de crear un modal independiente para cada caso.

5.6 Visualización de JSON individual y JSON general

Las funciones `mostrarModalJSONIndividual` y `mostrarModalJSONGeneral` reciben, respectivamente, un objeto estudiante o el arreglo completo, y utilizan `JSON.stringify` con una indentación de dos espacios para presentar la información de forma legible dentro del modal. Cuando el arreglo general está vacío, se valida y se muestra un mensaje alternativo en lugar de un JSON vacío.

```
function mostrarModalJSONIndividual(estudiante) {
  const contenidoJSON = document.getElementById("contenidoJSON");
  document.getElementById("modalJSONLabel").innerHTML = "JSON del estudiante";
  contenidoJSON.textContent = JSON.stringify(estudiante, null, 2);
  const modal = new bootstrap.Modal(document.getElementById("modalJSON"));
  modal.show();
}

function mostrarModalJSONGeneral() {
  const contenidoJSON = document.getElementById("contenidoJSON");
  document.getElementById("modalJSONLabel").innerHTML = "JSON de estudiantes
registrados";

  if (estudiantesRegistrados.length === 0) {
    contenidoJSON.textContent = "No existen estudiantes registrados.";
  } else {
    contenidoJSON.textContent = JSON.stringify(estudiantesRegistrados, null, 2);
  }

  const modal = new bootstrap.Modal(document.getElementById("modalJSON"));
  modal.show();
}
```

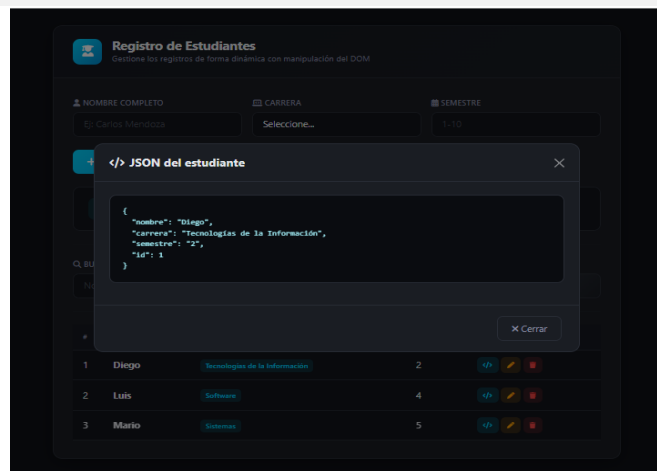


Figura 4. Modal de Bootstrap mostrando el JSON de un estudiante individual.

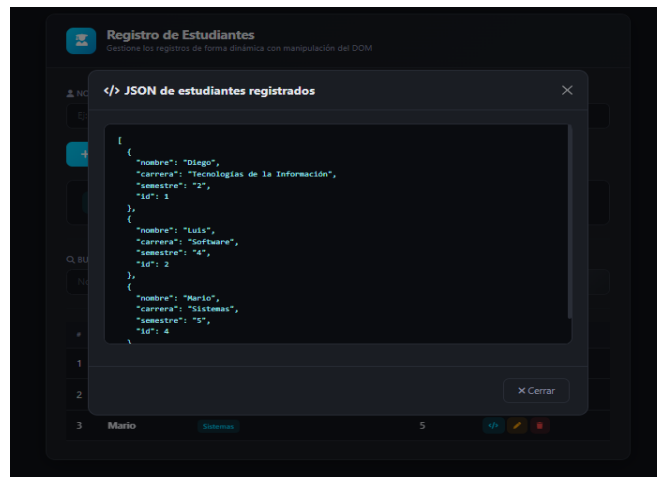


Figura 5. Modal de Bootstrap mostrando el JSON general de los estudiantes registrados.

5.7 Modo de edición de estudiantes

Al presionar el botón de editar, la función `editarEstudiante` carga los valores del estudiante seleccionado en el formulario y activa el modo de edición: el índice se guarda en `editandoIndex` y el texto del botón principal cambia de "Agregar estudiante" a "Guardar cambios", junto con un cambio de clase CSS que modifica su color.

```
function editarEstudiante(index) {  
  const estudiante = estudiantesRegistrados[index];  
  document.getElementById("nombre").value = estudiante.nombre;  
  document.getElementById("carrera").value = estudiante.carrera;  
  document.getElementById("semestre").value = estudiante.semestre;  
  
  editandoIndex = index;  
  btnAgregar.innerHTML = '<i class="fas fa-save me-1"></i>Guardar cambios';  
  btnAgregar.classList.remove("btn-accent");  
  btnAgregar.classList.add("btn-warning-custom");  
  
  mostrarToast("Editando estudiante: " + estudiante.nombre, "info");  
}
```

Análisis: la reutilización del mismo botón para agregar y para guardar cambios evita duplicar la interfaz; el estado se controla exclusivamente mediante la variable `editandoIndex` y las clases CSS aplicadas con `classList`, lo que mantiene sincronizados el texto, el color y el comportamiento del botón.

Registro de Estudiantes
Gestione los registros de forma dinámica con manipulación del DOM

Editar estudiante: Diego

NOMBRE COMPLETO: CARRERA: SEMESTRE:

Guardar cambios Limpiar tabla

Total de estudiantes registrados: NaN

Q BUSCAR ESTUDIANTE: FILTRAR POR CARRERA: </> JSON

#	NOMBRE	CARRERA	SEMESTRE	ACCIONES
1	Diego	Tecnologías de la Información	2	[iconos]
2	Luis	Software	4	[iconos]
3	Mario	Sistemas	5	[iconos]

Figura 6. Formulario cargado en modo edición, con el botón principal indicando "Guardar cambios".

Registro de Estudiantes
Gestione los registros de forma dinámica con manipulación del DOM

Estudiante editado correctamente

NOMBRE COMPLETO: CARRERA: SEMESTRE:

+ Agregar estudiante Limpiar tabla

Total de estudiantes registrados: NaN

Q BUSCAR ESTUDIANTE: FILTRAR POR CARRERA: </> JSON

#	NOMBRE	CARRERA	SEMESTRE	ACCIONES
1	Diego	Tecnologías de la Información	5	[iconos]
2	Luis	Software	4	[iconos]
3	Mario	Sistemas	5	[iconos]

Figura 7. Tabla actualizada tras confirmar la edición y notificación de éxito.

5.8 Eliminación de estudiantes con confirmación

La eliminación de un registro se realiza con splice sobre el arreglo, pero antes se solicita confirmación visual mediante SweetAlert2, evitando eliminaciones accidentales. Si el estudiante eliminado era el que estaba en edición, el formulario y el modo de edición se reinician.

```
function eliminarEstudiante(index) {
  const estudiante = estudiantesRegistrados[index];
  Swal.fire({
    title: "¿Eliminar estudiante?",
    text: 'Se eliminará a "' + estudiante.nombre + '" del registro.',
    icon: "question",
    showCancelButton: true,
    confirmButtonText: "Sí, eliminar",
    cancelButtonText: "Cancelar",
  }).then((result) => {
    if (result.isConfirmed) {
```

```

    estudiantesRegistrados.splice(index, 1);
    if (editandoIndex === index) {
        editandoIndex = null;
        limpiarFormulario();
    }
    renderizarTabla(estudiantesRegistrados);
    actualizarTotal();
    mostrarToast("Estudiante eliminado correctamente.", "danger");
}
});
}

```

Análisis: el uso de Swal.fire con showCancelButton retorna una promesa que solo ejecuta la eliminación si isConfirmed es verdadero, lo que separa la intención del usuario de la acción destructiva sobre el arreglo de datos.

5.9 Actualización del contador de estudiantes

El contador de estudiantes registrados se recalcula después de cada operación (agregar, editar, eliminar o limpiar) tomando como referencia la longitud actual del arreglo, y se anima con la librería anime.js para reforzar visualmente el cambio.

```

function actualizarTotal() {
    const totalEl = document.getElementById("totalEstudiantes");
    const valorActual = parseInt(totalEl.textContent) || 0;
    const valorFinal = estudiantesRegistrados.length;

    animate(
        { val: valorActual },
        {
            val: valorFinal,
            duration: 500,
            onUpdate: function (el) {
                totalEl.textContent = Math.round(el.val);
            },
        },
    );
}

```

Análisis: en lugar de asignar directamente el nuevo valor con textContent, se anima la transición numérica entre el valor anterior y el valor final, lo que hace más perceptible al usuario el cambio en la cantidad de estudiantes registrados.

5.10 Limpieza de tabla, arreglo y formulario

El botón "Limpiar tabla" solicita confirmación mediante SweetAlert2 y, al aceptarse, reinicia el arreglo de estudiantes, el contador, el estado de edición y el formulario, devolviendo el sistema a su estado inicial.

```

function limpiarTabla() {

```

```

Swal.fire({
  title: "¿Limpiar tabla?",
  text: "Se eliminarán todos los registros y se reiniciará el contador.",
  icon: "warning",
  showCancelButton: true,
  confirmButtonText: "Sí, limpiar",
  cancelButtonText: "Cancelar",
}).then((result) => {
  if (result.isConfirmed) {
    estudiantesRegistrados.length = 0;
    contadorEstudiantes = 0;
    editandoIndex = null;
    btnAgregar.innerHTML = '<i class="fas fa-plus me-1"></i>Agregar estudiante';
    btnAgregar.classList.remove("btn-warning-custom");
    btnAgregar.classList.add("btn-accent");
    renderizarTabla(estudiantesRegistrados);
    actualizarTotal();
    mostrarToast("Tabla limpiada correctamente.", "secondary");
  }
});
}

```

Análisis: fijar `estudiantesRegistrados.length = 0` vacía el arreglo sin perder la referencia original, lo cual es importante porque otras partes del sistema mantienen esa misma referencia; crear un arreglo nuevo con `[]` la habría roto.

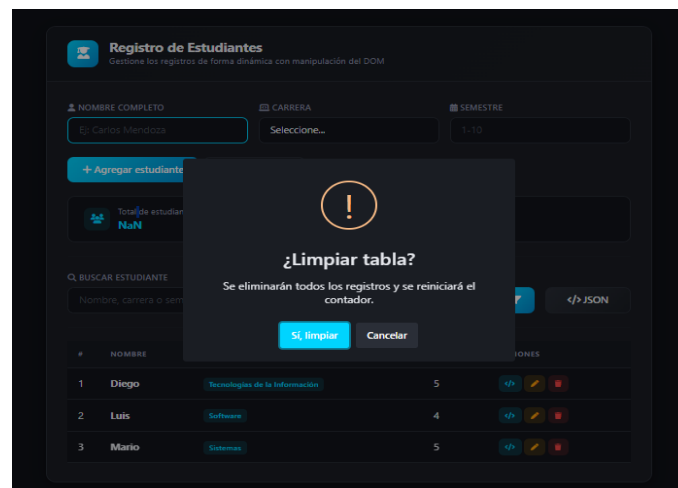


Figura 8. Cuadro de confirmación de SweetAlert2 antes de limpiar la tabla.

#	NOMBRE	CARRERA	SEMESTRE	ACCIONES
1	Diego	Tecnologías de la Información	5	[edit] [delete] [view]
2	Luis	Software	3	[edit] [delete] [view]
3	Mario	Sistemas	6	[edit] [delete] [view]

Figura 9. Tabla reconstruida con los estudiantes de prueba tras confirmar la operación.

5.11 Mensajes visuales con Toastify

Todas las operaciones del sistema (agregar, editar, eliminar, limpiar, validar y buscar) muestran una notificación mediante Toastify, cuyo color de fondo cambia según el tipo de mensaje: éxito, error, advertencia o información.

```
function mostrarToast(texto, tipo) {
  let colorFondo;
  switch (tipo) {
    case "success": colorFondo = "linear-gradient(135deg, #00b09b, #96c93d)"; break;
    case "danger": colorFondo = "linear-gradient(135deg, #ff416c, #ff4b2b)"; break;
    case "warning": colorFondo = "linear-gradient(135deg, #f7971e, #ffd200)"; break;
    case "info": colorFondo = "linear-gradient(135deg, #00d4ff, #0088cc)"; break;
    default: colorFondo = "linear-gradient(135deg, #636e72, #b2bec3)";
  }

  Toastify({
    text: texto,
    duration: 3000,
    gravity: "top",
    position: "right",
    style: { background: colorFondo, borderRadius: "10px" },
  }).showToast();
}
```

Análisis: centralizar las notificaciones en una única función parametrizada por tipo evita repetir la configuración de Toastify en cada punto del código y permite mantener un lenguaje visual consistente en todo el sistema.

5.12 Búsqueda de estudiantes por texto

El campo de búsqueda captura el texto ingresado, lo normaliza a minúsculas y filtra el arreglo `estudiantesRegistrados` comparando el nombre, la carrera y el semestre convertido a texto. El resultado del filtro se vuelve a pasar a `renderizarTabla`, sin modificar el arreglo original.

```
function buscarEstudiantes() {
```



```

const texto = document.getElementById("buscarTexto").value.trim().toLowerCase();

if (texto === "") {
  mostrarToast("Ingrese un texto para buscar.", "warning");
  return;
}

const resultados = estudiantesRegistrados.filter((estudiante) => {
  return (
    estudiante.nombre.toLowerCase().includes(texto) ||
    estudiante.carrera.toLowerCase().includes(texto) ||
    estudiante.semestre.toString().includes(texto)
  );
});

renderizarTabla(resultados);

if (resultados.length === 0) {
  mostrarToast("No se encontraron resultados.", "warning");
} else {
  mostrarToast("Se encontraron " + resultados.length + " resultado(s).", "info");
}
}

```

Análisis: el método filter genera un nuevo arreglo con los estudiantes que cumplen la condición, sin alterar estudiantesRegistrados; esto permite que, al limpiar la búsqueda, la tabla pueda reconstruirse nuevamente con la totalidad de los registros originales.

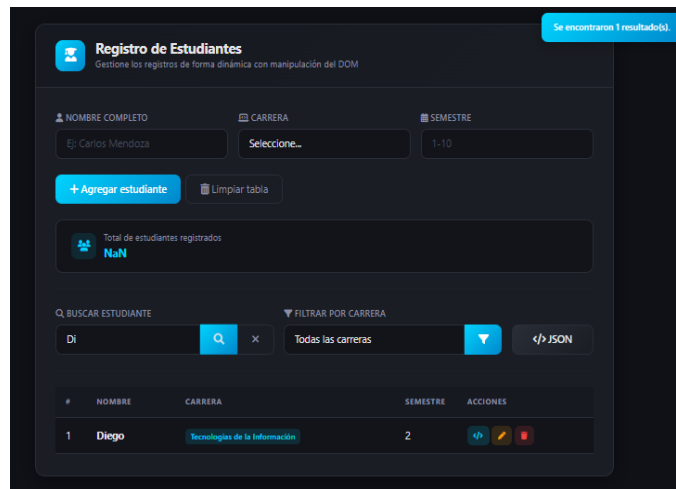


Figura 10. Resultado de la búsqueda por texto sobre el arreglo de estudiantes.

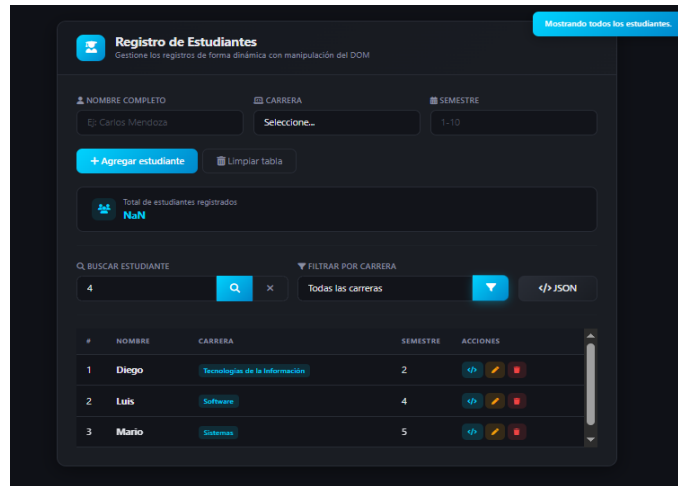


Figura 11. Restablecimiento de la tabla completa tras limpiar el campo de búsqueda.

5.13 Filtro por carrera

El select de filtro toma el valor seleccionado y compara la carrera de cada estudiante con `estudiante.carrera === carreraSeleccionada`. Cuando la opción elegida es "Todas", se restaura la tabla completa sin aplicar ningún filtro.

```
function filtrarPorCarrera() {
  const carreraSeleccionada = document.getElementById("filtroCarrera").value;

  if (carreraSeleccionada === "Todas") {
    renderizarTabla(estudiantesRegistrados);
    mostrarToast("Mostrando todos los estudiantes.", "info");
    return;
  }

  const resultados = estudiantesRegistrados.filter((estudiante) => {
    return estudiante.carrera === carreraSeleccionada;
  });

  renderizarTabla(resultados);

  if (resultados.length === 0) {
    mostrarToast("No hay estudiantes en la carrera seleccionada.", "warning");
  } else {
    mostrarToast("Se encontraron " + resultados.length + " estudiante(s) en " +
      carreraSeleccionada + ".", "info");
  }
}
```

Análisis: al igual que en la búsqueda por texto, el filtrado se realiza directamente sobre el arreglo de datos y no sobre las filas ya dibujadas en el HTML, lo que garantiza resultados consistentes independientemente del estado previo de la tabla.

Registro de Estudiantes
Gestione los registros de forma dinámica con manipulación del DOM

Se encontraron 1 estudiante(s) en Tecnologías de la Información

NOMBRE COMPLETO: Ej: Carlos Mendoza
CARRERA: Seleccione...
SEMESTRE: 1-10

+ Agregar estudiante Limpiar tabla

Total de estudiantes registrados: NaN

BUSCAR ESTUDIANTE: Nombre, carrera o semestre...
 FILTRAR POR CARRERA: Tecnologías de la Información
 </> JSON

#	NOMBRE	CARRERA	SEMESTRE	ACCIONES
1	Diego	Tecnologías de la Información	2	

Figura 12. Filtro de estudiantes aplicado sobre la carrera Tecnologías de la Información.

5.14 Pruebas del sistema con estudiantes registrados

Se registraron varios estudiantes de prueba para verificar el funcionamiento integral del sistema: registro, visualización de JSON individual y general, búsqueda, filtrado, edición, eliminación, limpieza y actualización del contador. Todas las pruebas mostraron el comportamiento esperado en cada funcionalidad.

Registro de Estudiantes
Gestione los registros de forma dinámica con manipulación del DOM

NOMBRE COMPLETO: Ej: Carlos Mendoza
CARRERA: Seleccione...
SEMESTRE: 1-10

+ Agregar estudiante Limpiar tabla

Total de estudiantes registrados: 1

BUSCAR ESTUDIANTE: Nombre, carrera o semestre...
 FILTRAR POR CARRERA: Todas las carreras
 </> JSON

Figura 13. Vista responsiva del formulario de registro en un ancho de pantalla reducido.

6. Librería externa: SweetAlert2 y Toastify

Para la actividad complementaria del laboratorio se integraron dos librerías externas de interacción visual: SweetAlert2 y Toastify. Ambas se incorporaron al proyecto mediante enlaces CDN dentro del archivo index.html, sin requerir instalación local.

```
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/toastify-
js/src/toastify.min.css">
<script src="https://cdn.jsdelivr.net/npm/toastify-js"></script>
```

SweetAlert2 se utilizó para mostrar cuadros de confirmación antes de ejecutar acciones irreversibles, específicamente la eliminación de un estudiante y la limpieza completa de la tabla. Su método `Swal.fire` retorna una promesa que permite ejecutar la acción únicamente si el usuario confirma, reemplazando al `confirm()` nativo del navegador con una interfaz más clara y personalizable en color y texto.

Toastify se utilizó para mostrar notificaciones breves y no bloqueantes cada vez que se agrega, edita, elimina o limpia información, así como en las validaciones de formulario y de búsqueda. A diferencia de un `alert()` del navegador, Toastify no detiene la ejecución del script ni exige una acción del usuario para continuar, y permite personalizar la posición, la duración y el color de fondo del mensaje.

Frente a los mensajes básicos del navegador, ambas librerías aportan una mejora significativa en la experiencia de usuario: los mensajes se integran visualmente con el tema oscuro de la aplicación, no interrumpen el flujo de trabajo y comunican de forma más clara el resultado de cada acción mediante colores semánticos (verde para éxito, rojo para eliminación, amarillo para advertencias y celeste para información).

7. Criterio personal

El desarrollo de este laboratorio permitió comprender de forma práctica cómo el DOM actúa como puente entre el HTML estático y la lógica dinámica de JavaScript. Trabajar con un arreglo de objetos como única fuente de verdad para la tabla dejó claro que la interfaz debe ser siempre una representación del estado de los datos, y no al revés, evitando así inconsistencias entre lo que el usuario ve y lo que realmente está almacenado.

La implementación del modo de edición reforzó el manejo de estados mediante variables de control como `editandoIndex`, mientras que la eliminación con confirmación y la limpieza general evidenciaron la importancia de proteger al usuario de acciones irreversibles. El uso de modales de Bootstrap para mostrar información en formato JSON facilitó comprender cómo estructurar y depurar datos complejos de manera visual.

Finalmente, la integración de SweetAlert2 y Toastify demostró el valor de las librerías externas para mejorar la experiencia de usuario sin necesidad de escribir componentes visuales desde cero, y sentó una base sólida para continuar avanzando hacia interfaces web más completas e interactivas.

8. Conclusiones

- DOM permite construir interfaces completamente dinámicas a partir de un arreglo de objetos como única fuente de datos. Al aplicar los métodos `createElement`, `appendChild`, `textContent` e `innerHTML`, logré generar una tabla con columna de acciones sin necesidad de frameworks adicionales, entendiendo que cada nodo se crea en memoria y se inserta en el árbol del documento de forma programática.
- El manejo de estado mediante variables de control, como `editandoIndex`, resulta esencial para diferenciar entre el registro de un nuevo estudiante y la modificación de uno existente. Sin esta variable, no habría forma de saber si el botón principal debe crear o actualizar, lo cual me permitió reutilizar la misma función `agregarEstudiante` para ambas operaciones.

9. Recomendaciones

- Validar siempre los campos del formulario antes de insertar o actualizar un registro en el arreglo de datos. Centralizar la lógica de notificaciones en una única función parametrizada, como `mostrarToast`, para mantener consistencia visual.
- Utilizar confirmaciones visuales, como las de `SweetAlert2`, en toda acción que elimine o reinicie información del usuario. Probar el sistema con datos variados, incluyendo casos vacíos, antes de dar por finalizada una funcionalidad.
- Documentar cada función con comentarios breves que faciliten su mantenimiento y comprensión posterior. Mantener la separación entre lógica de negocio (arreglo de datos) y lógica de presentación.

10. Bibliografía

Modelo de Objetos del Documento (DOM) - API web | MDN. (2026b, marzo 31).

https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model

JSON.stringify() - JavaScript | MDN. (2025, 31 marzo).

https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

Contributors, M. o. J. T. A. B. (2022). *Modal*. <https://getbootstrap.com/docs/5.3/components/modal/>

SweetAlert2. (2021) A Beautiful, Responsive, Customizable And Accessible (WAI-ARIA) Replacement For JavaScript's Popup Boxes. <https://sweetalert2.github.io/>

Apvarun. (2020). *GitHub - apvarun/toastify-js: Pure JavaScript library for better notification messages.*

GitHub. <https://github.com/apvarun/toastify-js>